

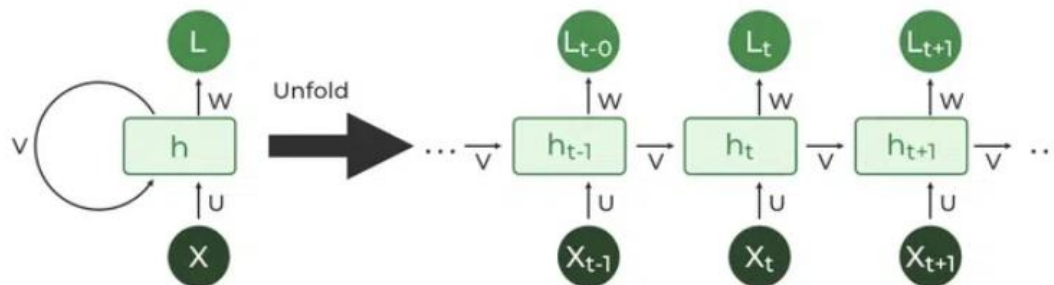
UNIT III

RECURRENT NEURAL NETWORK

Syllabus : Recurrent Neural Networks: Introduction – Recursive Neural Networks – Bidirectional RNNs – Deep Recurrent Networks - Complete Autoencoder; Regularized Autoencoder; Stochastic Encoders and Decoders, Contractive Encoders – Language Modelling – Sequence to sequence learning – Speech Recognition

RECURRENT NEURAL NETWORKS

Recurrent Neural Network(RNN) is a type of Neural Network where the output from the previous step is fed as input to the current step. In traditional neural networks, all the inputs and outputs are independent of each other. Still, in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words. Thus RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is its Hidden state, which remembers some information about a sequence. The state is also referred to as Memory State since it remembers the previous input to the network. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.



Recurrent Neural Network

Artificial neural networks that do not have looping nodes are called feed forward neural networks. Because all information is only passed forward, this kind of neural network is also referred to as a multi-layer neural network.

Information moves from the input layer to the output layer – if any hidden layers are present – unidirectionally in a feedforward neural network. These networks are appropriate for image classification tasks, for example, where input and output are independent. Nevertheless, their inability to retain previous inputs automatically renders them less useful for sequential data analysis.

The fundamental processing unit in a Recurrent Neural Network (RNN) is a Recurrent Unit, which is not explicitly called a “Recurrent Neuron.” This unit has the unique ability to maintain a hidden state, allowing the network to capture sequential dependencies by remembering previous inputs while processing.

Recurrent Neural Network Architecture

The hidden state update in an RNN can be expressed mathematically as:

$$H_i = f(W_h H_{i-1} + W_x X_i + b)$$

where:

- H_i is the hidden state at time step i .
- W_h and W_x are weight matrices for the hidden state and the input, respectively.
- b is a bias vector.
- f is an activation function, such as tanh or ReLU.

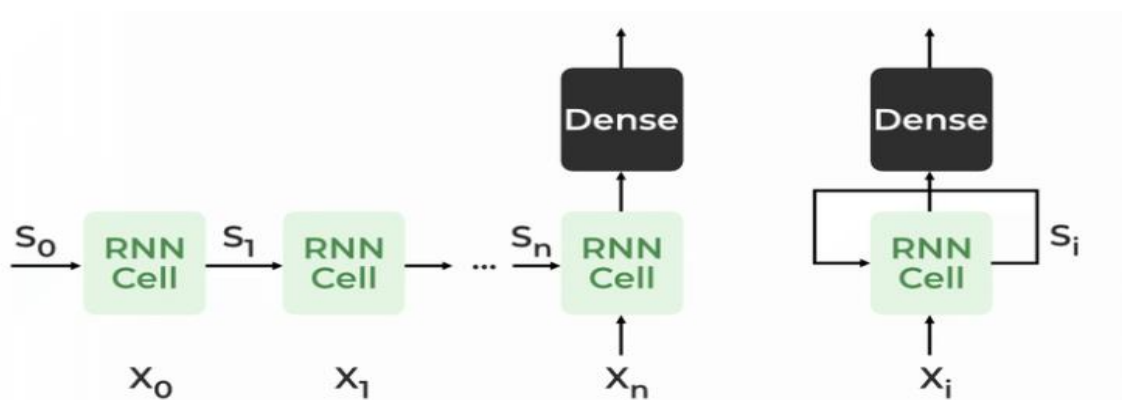
The output Y_i at each time step can be computed as:

$$Y_i = g(W_y H_i + c)$$

where:

- W_y is the weight matrix for the output.
- c is a bias vector for the output.
- g is an activation function or softmax for classification tasks.

RNNs are particularly useful for tasks involving sequential data, such as time series prediction, natural language processing, and speech recognition, where the temporal dependencies between inputs need to be captured. However, they can struggle with long-range dependencies due to issues like vanishing and exploding gradients, which is why architectures like LSTMs (Long Short-Term Memory) and GRUs (Gated Recurrent Units) were developed to improve their performance.



Recurrent Neural Architecture

How does RNN work?

The Recurrent Neural Network consists of multiple fixed activation function units, one for each time step. Each unit has an internal state which is called the hidden state of the unit. This hidden state signifies the past knowledge that the network currently holds at a given time step. This hidden state is updated at every time step to signify the change in the knowledge of the network about the past. The hidden state is updated using the following recurrence relation:-

The formula for calculating the current state:

$$h_t = f(h_{t-1}, x_t)$$

where,

- h_t -> current state
- h_{t-1} -> previous state
- x_t -> input state

Formula for applying Activation function(tanh)

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

where,

- w_{hh} -> weight at recurrent neuron
- w_{xh} -> weight at input neuron

The formula for calculating output:

$$y_t = W_{hy}h_t$$

- Y_t -> output
- W_{hy} -> weight at output layer

Training through RNN

1. A single-time step of the input is provided to the network.
2. Then calculate its current state using a set of current input and the previous state.
3. The current h_t becomes h_{t-1} for the next time step.
4. One can go as many time steps according to the problem and join the information from all the previous states.
5. Once all the time steps are completed the final current state is used to calculate the output.

6. The output is then compared to the actual output i.e the target output and the error is generated.
7. The error is then back-propagated to the network to update the weights and hence the network (RNN) is trained using Backpropagation through time.

Advantages and Disadvantages of Recurrent Neural Network

Advantages

1. An RNN remembers each and every piece of information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called Long Short Term Memory.
2. Recurrent neural networks are even used with convolutional layers to extend the effective pixel neighborhood.

Disadvantages

1. Gradient vanishing and exploding problems.
2. Training an RNN is a very difficult task.
3. It cannot process very long sequences if using tanh or relu as an activation function.

Applications of Recurrent Neural Network

1. Language Modelling and Generating Text
2. Speech Recognition
3. Machine Translation
4. Image Recognition, Face detection
5. Time series Forecasting

RECURSIVE NEURAL NETWORKS

Recursive Neural Networks are a type of neural network architecture that is specially designed to process hierarchical structures and capture dependencies within recursively structured data. Unlike traditional feedforward neural networks (RNNs), Recursive Neural Networks or RvNN can efficiently handle tree-structured inputs which makes them suitable for tasks involving nested and hierarchical relationships.

RvNN is a special Deep Learning which has the ability to operate on structured input data like parse trees in natural language processing or molecular structures in chemistry. The network processes the input recursively, combining information from child nodes to form representations for parent nodes. RvNN mainly used in some NLP tasks like sentiment analysis etc. by processing data which is in the format of parse tree. RvNN processes parse trees by assigning vectors to each word or subphrase based on the information from its children which allows the network to capture hierarchical relationships and dependencies within the sentence.

Working Principals of RvNN

Some of the key-working principals of RvNN is discussed below:

1. **Recursive Structure Handling:** RvNN is designed to handle recursive structures which means it can naturally process hierarchical relationships in data by combining information from child nodes to form representations for parent nodes.
2. **Parameter Sharing:** RvNN often uses shared parameters across different levels of the hierarchy which enables the model to generalize well and learn from various parts of the input structure.
3. **Tree Traversal:** RvNN traverses the tree structure in a bottom-up or top-down manner by simultaneously updating node representations based on the information gathered from their children.
4. **Composition Function:** The composition function in RvNN combines information from child nodes to create a representation for the parent node. This function is crucial in capturing the hierarchical relationships within the data.

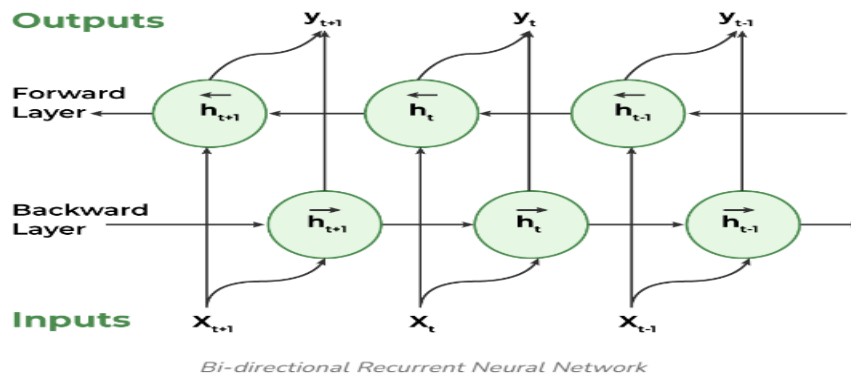
BIDIRECTIONAL RNNs

A Bidirectional Recurrent Neural Network (BRNN) is designed to process sequential data by using both past and future context for making predictions, unlike traditional Recurrent Neural Networks (RNNs) that only use past context. A BRNN achieves this by having two separate recurrent hidden layers: one processes the input sequence in the forward direction, while the other processes it in the backward direction. The outputs from these hidden layers are then combined to make the final prediction. These recurrent hidden layers can be built using any RNN cell, such as Long Short-Term Memory (LSTM) or Gated Recurrent Unit (GRU).

In the forward direction, a BRNN functions like a regular RNN, updating its hidden state based on the current input and the previous hidden state at each time step. The backward hidden layer processes the sequence in reverse, updating the hidden state based on the current input and the hidden state from the next time step.

BRNNs improve accuracy compared to traditional unidirectional RNNs because they can consider information from both directions, allowing them to use both past and future contexts. Having two separate hidden layers also acts as a form of model regularization, providing more data to the final prediction layer.

During training, BRNNs use the backpropagation through time technique to compute gradients for both the forward and backward passes. At inference time, the BRNN processes the input sequence in a single forward pass and makes predictions based on the combined outputs of the two hidden layers.



Working of Bidirectional Recurrent Neural Network

1. **Inputting a sequence:** A sequence of data points, each represented as a vector with the same dimensionality, are fed into a BRNN. The sequence might have different lengths.
2. **Dual Processing:** Both the forward and backward directions are used to process the data. On the basis of the input at that step and the hidden state at step $t-1$, the hidden state at time step t is determined in the forward direction. The input at step t and the hidden state at step $t+1$ are used to calculate the hidden state at step t in a reverse way.
3. **Computing the hidden state:** A non-linear activation function on the weighted sum of the input and previous hidden state is used to calculate the hidden state at each step. This creates a memory mechanism that enables the network to remember data from earlier steps in the process.
4. **Determining the output:** A non-linear activation function is used to determine the output at each step from the weighted sum of the hidden state and a number of output weights. This output has two options: it can be the final output or input for another layer in the network.
5. **Training:** The network is trained through a supervised learning approach where the goal is to minimize the discrepancy between the predicted output and the actual output. The network adjusts its weights in the input-to-hidden and hidden-to-output connections during training through backpropagation.

Applications of Bidirectional Recurrent Neural Network

Bi-RNNs have been applied to various natural language processing (NLP) tasks, including:

1. **Sentiment Analysis:** By taking into account both the prior and subsequent context, BRNNs can be utilized to categorize the sentiment of a particular sentence.
2. **Named Entity Recognition:** By considering the context both before and after the stated thing, BRNNs can be utilized to identify those entities in a sentence.
3. **Part-of-Speech Tagging:** The classification of words in a phrase into their corresponding parts of speech, such as nouns, verbs, adjectives, etc., can be done using BRNNs.

4. **Machine Translation:** BRNNs can be used in encoder-decoder models for machine translation, where the decoder creates the target sentence and the encoder analyses the source sentence in both directions to capture its context.
5. **Speech Recognition:** When the input voice signal is processed in both directions to capture the contextual information, BRNNs can be used in automatic speech recognition systems.

Advantages of Bidirectional RNN

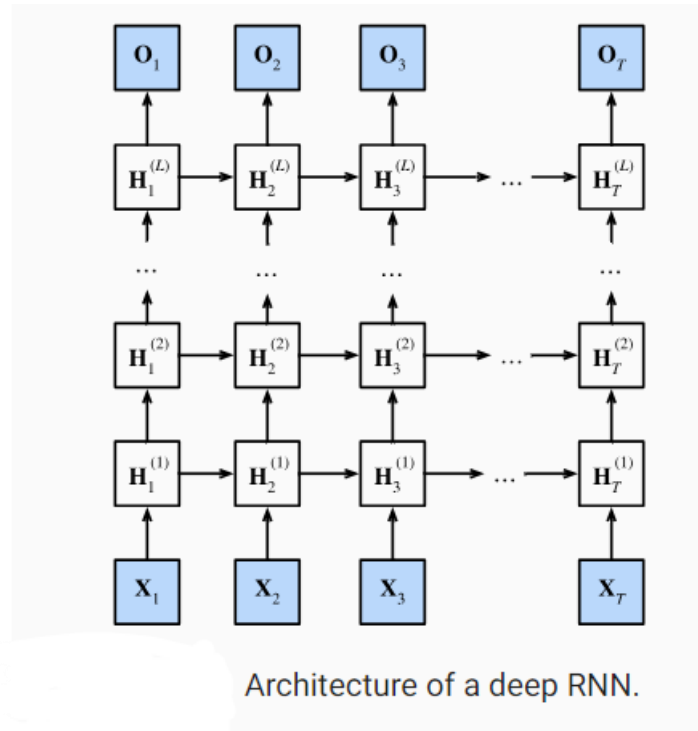
- **Context from both past and future:** With the ability to process sequential input both forward and backward, BRNNs provide a thorough grasp of the full context of a sequence. Because of this, BRNNs are effective at tasks like sentiment analysis and speech recognition.
- **Enhanced accuracy:** BRNNs frequently yield more precise answers since they take both historical and upcoming data into account.
- **Efficient handling of variable-length sequences:** When compared to conventional RNNs, which require padding to have a constant length, BRNNs are better equipped to handle variable-length sequences.
- **Resilience to noise and irrelevant information:** BRNNs may be resistant to noise and irrelevant data that are present in the data. This is so because both the forward and backward paths offer useful information that supports the predictions made by the network.
- **Ability to handle sequential dependencies:** BRNNs can capture long-term links between sequence pieces, making them extremely adept at handling complicated sequential dependencies.

Disadvantages of Bidirectional RNN

- **Computational complexity:** Given that they analyze data both forward and backward, BRNNs can be computationally expensive due to the increased amount of calculations needed.
- **Long training time:** BRNNs can also take a while to train because there are many parameters to optimize, especially when using huge datasets.

DEEP RECURRENT NETWORKS

Deep RNNs (Recurrent Neural Networks) are advanced computer programs that learn to identify patterns in sequential data, such as words in a sentence or musical notes in a song. They operate by processing data through multiple layers, each layer building a more detailed understanding of the information. This layered approach helps Deep RNNs to recognize complex relationships between different parts of the data and make accurate predictions about future sequences.



These networks are widely used in real-world applications, including speech recognition systems like Siri and Alexa, language translation software, and autonomous vehicles. Deep RNNs are especially valuable when dealing with large amounts of sequential data, such as teaching a computer to understand human language. By effectively handling sequential data and capturing intricate relationships between inputs and outputs, Deep RNNs have become essential tools in fields ranging from speech recognition and natural language processing to music generation and self-driving cars.

What is a Deep RNN?

A Deep RNN is a neural network architecture featuring multiple layers of recurrent units. These networks are designed to manage sequential data, such as time series or natural language, by keeping an internal memory of past inputs. In a Deep RNN, the output from one layer of recurrent units is fed into the next layer, enabling the network to understand more complex relationships between input and output sequences. The number of layers and hidden units in a Deep RNN can be adjusted based on the complexity of the problem.

Deep RNNs have demonstrated superior performance over single-layer RNNs and shallow neural networks in various applications, including natural language processing, speech recognition, image captioning, and music generation.

Real-Life Examples of Deep RNNs:

- **Speech Recognition:** Deep RNNs power speech recognition systems like Google's Speech API, Amazon's Alexa, and Apple's Siri, which transform speech signals into text.
- **Natural Language Processing (NLP):** These networks are used in language translation, sentiment analysis, and text classification, such as in Google Translate for translating text between languages.

- **Music Generation:** Deep RNNs are employed to create music, as seen in Magenta's MusicVAE, which generates melodies and harmonies.
- **Image Captioning:** In systems like Google's Show and Tell, Deep RNNs generate descriptive captions for images.
- **Autonomous Driving:** Companies like Waymo use Deep RNNs to predict the behavior of other vehicles on the road in autonomous driving systems.

Steps to Develop a Deep RNN Application:

Creating a complete Deep RNN application involves several stages, including data preparation, model architecture design, model training, and deployment. Here's an outline for developing a Deep RNN application for sentiment analysis:

1. **Data Preparation:** Start by collecting and preprocessing data, such as a dataset of text reviews labeled with positive or negative sentiment. Clean, tokenize, and convert the text into a numerical format using libraries like NLTK or spaCy in Python.
2. **Model Architecture Design:** Design the Deep RNN architecture by deciding on the number of layers, hidden units, and type of recurrent unit (e.g., LSTM or GRU). Determine how to handle input and output sequences using methods like padding or truncation.
3. **Training the Model:** Train the model with the preprocessed data, splitting it into training and validation sets. Use an optimization algorithm like stochastic gradient descent and set hyperparameters such as learning rate and batch size.
4. **Evaluating the Model:** After training, assess the model's performance on a separate test set using metrics like accuracy, precision, recall, and F1 score.
5. **Deploying the Model:** Finally, deploy the trained model in a production environment for real-time sentiment classification, potentially integrating it into a web application or API.

COMPLETE AUTOENCODER

An Autoencoder is a type of neural network that can learn to reconstruct images, text, and other data from compressed versions of themselves.

An Autoencoder consists of three layers:

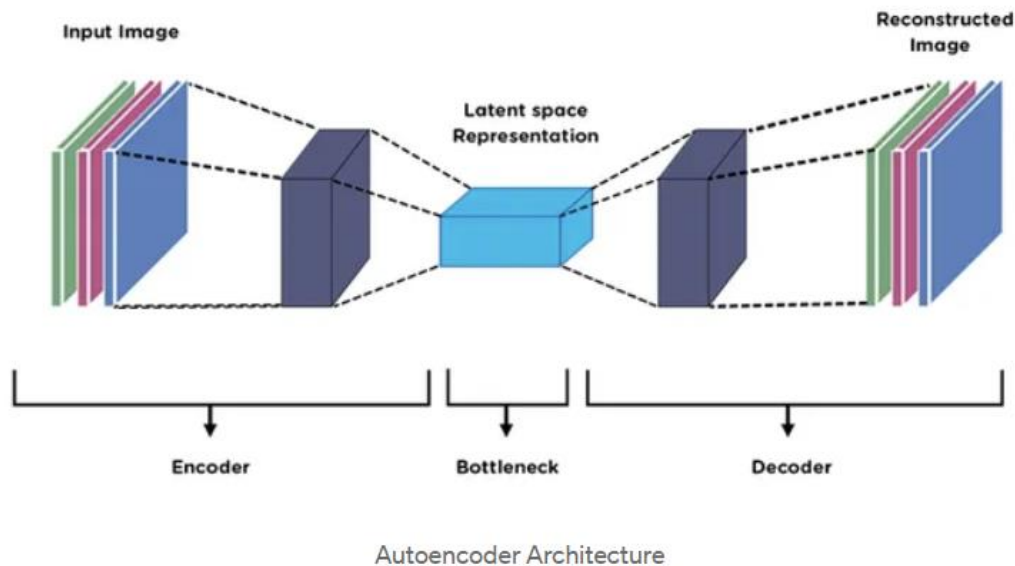
1. Encoder
2. Code
3. Decoder

The Encoder layer compresses the input image into a latent space representation. It encodes the input image as a compressed representation in a reduced dimension.

The compressed image is a distorted version of the original image.

The Code layer represents the compressed input fed to the decoder layer.

The decoder layer decodes the encoded image back to the original dimension. The decoded image is reconstructed from latent space representation, and it is reconstructed from the latent space representation and is a lossy reconstruction of the original image.



Components of a Complete Autoencoder:

1. Encoder:

- The encoder compresses the input data into a latent-space representation (also called the bottleneck).
- The encoder function $h=f(x)$ maps the input x to the hidden layer or latent space h .

2. Latent Space (Bottleneck):

- This is the middle layer of the network, where the data is represented in a lower dimension.
- The bottleneck forces the autoencoder to learn the most important features of the input data, as it has to compress the data into this small space.

3. Decoder:

- The decoder reconstructs the input data from the latent representation.
- The decoder function $\hat{x}=g(h)$ maps the latent space representation h back to the original data space $\hat{x}$, attempting to recreate the input.

4. Loss Function:

- The autoencoder is trained to minimize the difference between the input data x and the reconstructed data $\hat{x}$.

- Common loss functions include Mean Squared Error (MSE) for continuous data and Binary Cross-Entropy for binary data.

Properties of Autoencoders

- Autoencoders are data-specific, which means that they will only be able to compress data similar to what they have been trained on. Example, an autoencoder trained on pictures of faces would do a rather poor job of compressing pictures of trees, because the features it would learn would be face-specific.
- Autoencoders are lossy, which means that the decompressed outputs will be degraded compared to the original inputs.
- Autoencoders are learned automatically from data examples, which is a useful property: it means that it is easy to train specialized instances of the algorithm that will perform well on a specific type of input. It doesn't require any new engineering, just appropriate training data.

REGULARIZED AUTOENCODER

Regularization is a technique to constrain the model's complexity and prevent overfitting. By introducing additional terms to the loss function, it encourages the model to learn more generalizable features.

Types of Regularized Autoencoders:

- **SPARSE AUTOENCODER:**
 - A sparse autoencoder includes a sparsity constraint on the activation of the neurons in the hidden layer. This constraint forces the autoencoder to learn a compressed and more interpretable representation of the input data, often capturing the most relevant features.

How it Works

1. **Encoding:** The input data is fed into the encoder network, which compresses the data into a lower-dimensional latent space representation.
2. **Sparsity Constraint:** The activations of the hidden layer neurons are penalized if they deviate from a desired sparsity level. This encourages the network to use only a few neurons for each input.
3. **Decoding:** The compressed representation is fed into the decoder network, which attempts to reconstruct the original input data.
4. **Loss Function:** The overall loss function consists of two components:
 - Reconstruction error: Measures the difference between the original input and the reconstructed output.
 - Sparsity penalty: Enforces the sparsity constraint on the hidden layer activations.

Advantages of Sparse Autoencoders

- **Improved generalization:** By learning more discriminative features, sparse autoencoders are less prone to overfitting.
- **Better feature representation:** The sparse coding encourages the network to capture essential patterns in the data.
- **Potential for interpretability:** Sparse representations can be more easily understood and analyzed.

Applications

- **Feature learning:** Extracting meaningful features from unlabeled data.
- **Dimensionality reduction:** Compressing data while preserving essential information.
- **Anomaly detection:** Identifying unusual data points based on reconstruction errors.
- **Image denoising:** Removing noise from images.
- **DENOISING AUTOENCODER (DAE):**
 - In a denoising autoencoder, noise is added to the input data, and the autoencoder is trained to reconstruct the original, noise-free input. This regularization helps the model to learn robust features that are invariant to small perturbations in the input.
 - A Denoising Autoencoder (DAE) is a variant of the standard autoencoder designed to reconstruct the original input from a corrupted or noisy version. This forces the network to learn robust features that are invariant to noise, improving its ability to generalize and extract meaningful information from the data.

How it Works

- **Corrupting the Input:** Noise is intentionally added to the input data. This can be Gaussian noise, salt-and-pepper noise, or other forms of corruption.
- **Encoding:** The noisy input is fed into the encoder, which maps it to a lower-dimensional latent space representation.
- **Decoding:** The encoder's output is passed to the decoder, which aims to reconstruct the original, clean input data.
- **Loss Function:** The network is trained to minimize the difference between the reconstructed output and the original, clean input. This forces the autoencoder to learn to remove the noise and recover the underlying structure of the data.

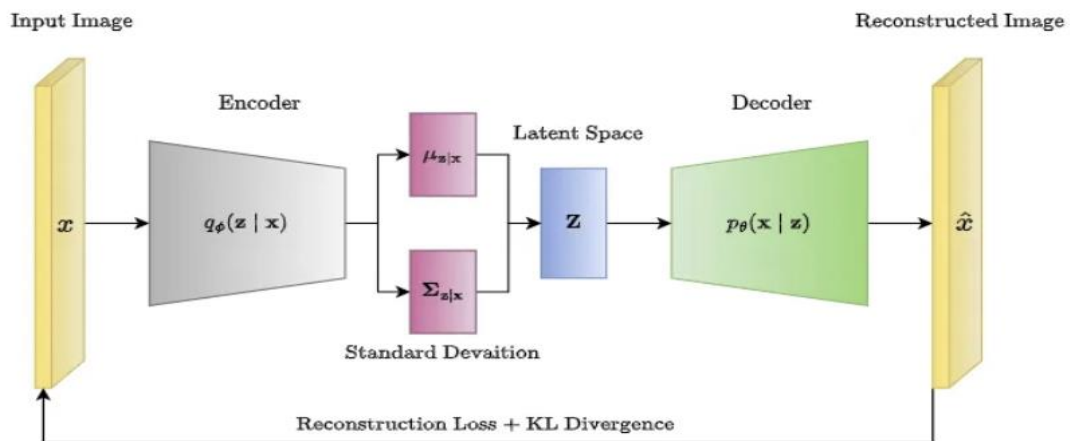
Applications

- **Image Denoising:** Removing noise from images.
- **Image Inpainting:** Filling missing parts of images.

- **Feature Learning:** Extracting meaningful features from noisy data.
- **Anomaly Detection:** Identifying abnormal data points based on reconstruction errors.
- **CONTRACTIVE AUTOENCODER (CAE):**
 - A contractive autoencoder minimizes the sensitivity of the learned features to small variations in the input. It does so by adding a penalty to the loss function, which is proportional to the norm of the Jacobian matrix of the encoder function. This encourages the model to learn features that are more stable and less likely to change with minor changes in input.
- **VARIATIONAL AUTOENCODER (VAE):**
 - In a variational autoencoder, the encoder produces parameters for a probability distribution (typically a Gaussian). The latent space is sampled from this distribution, and the decoder attempts to reconstruct the data from these samples. The loss function includes a regularization term (KL divergence) that encourages the latent space to follow a specific distribution, making the model suitable for generating new data.
 - A Variational Autoencoder (VAE) is a generative model that learns a probabilistic representation of the input data. Unlike traditional autoencoders that output a single point in the latent space, VAEs output a distribution. This probabilistic nature allows for better generation of new data points.

How it Works

- **Encoder:** The encoder maps the input data to a latent space, but instead of producing a single point, it outputs the parameters of a probability distribution (typically a Gaussian).
- **Sampling:** A sample is drawn from the learned distribution in the latent space.
- **Decoder:** The sampled point is fed into the decoder, which reconstructs the original input data.
- **Loss Function:** The VAE minimizes two loss terms:
 - Reconstruction loss: Measures the difference between the original input and the reconstructed output.
 - KL divergence: Encourages the learned distribution in the latent space to be close to a prior distribution (usually a standard normal distribution). This regularizes the latent space and prevents it from collapsing.



Key Components

- **Latent Space:** A lower-dimensional representation of the input data.
- **Encoder:** Maps input data to a probability distribution in the latent space.
- **Decoder:** Maps points from the latent space back to the original data space.
- **Reparameterization Trick:** A technique used to backpropagate through the sampling process.

Applications

- **Image Generation:** Generating new images similar to a given dataset.
- **Data Imputation:** Filling missing values in data.
- **Anomaly Detection:** Identifying unusual data points.
- **Drug Discovery:** Generating new molecular structures.

STOCHASTIC ENCODERS AND DECODERS

Stochastic Encoders and Decoders are used in the context of autoencoders, particularly in Variational Autoencoders (VAEs) and other probabilistic models. These components introduce randomness into the encoding and decoding processes, which can be beneficial for tasks like generative modeling, uncertainty estimation, and learning more robust representations.

Key Concepts:

1. Stochastic Encoder:

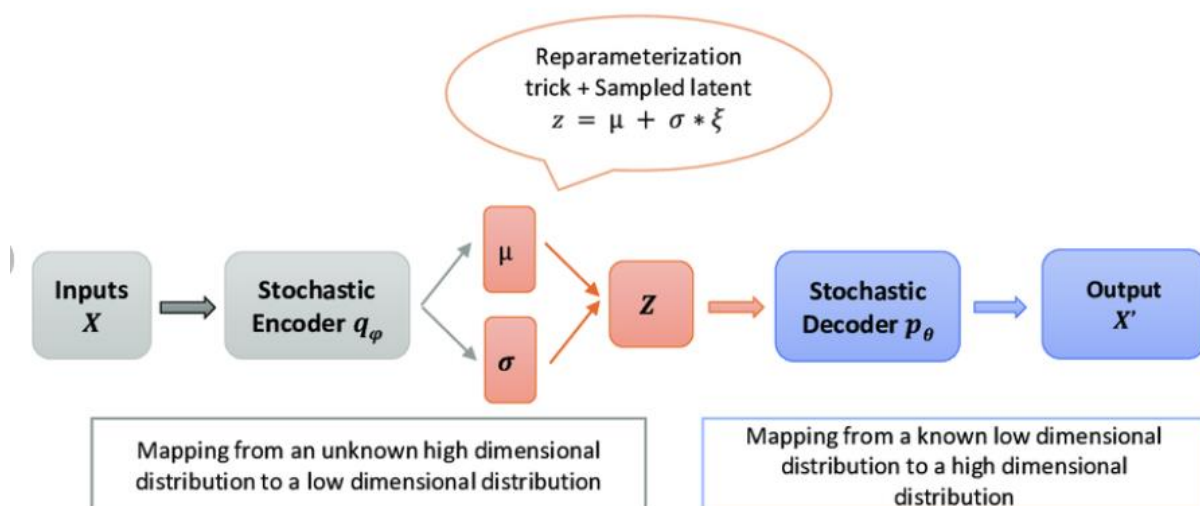
- In a stochastic encoder, the encoding process involves sampling from a probability distribution rather than producing a single deterministic output.
- For example, instead of mapping an input x to a single latent vector z , the encoder maps x to the parameters of a probability distribution (usually a Gaussian), such as a mean $\mu(x)$ and a variance $\sigma^2(x)$.
- The latent vector z is then sampled from this distribution:

$$z \sim \mathcal{N}(\mu(x), \sigma^2(x))$$

- This introduces stochasticity (randomness) in the representation, which can help the model learn a more diverse set of latent features and improve its ability to generalize.

2. Stochastic Decoder:

- A stochastic decoder takes a sampled latent vector z and produces a distribution over possible reconstructions $\hat{x}$ rather than a single deterministic reconstruction.
- Similar to the encoder, the decoder might output parameters of a probability distribution (e.g., mean and variance for a Gaussian), and the final output is sampled from this distribution.
- The stochasticity in the decoder can make the model more flexible in generating diverse outputs, which is particularly useful in generative tasks.



CONTRACTIVE ENCODERS

A contractive autoencoder is considered an unsupervised deep learning technique. It helps a neural network to encode unlabeled training data. The idea behind that is to make the autoencoders robust small changes in the training dataset.

We use autoencoders to learn a representation, or encoding, for a set of unlabeled data. It is usually the first step towards dimensionality reduction or generating new data models. Contractive autoencoder targets to learn invariant representations to unimportant transformations for the given data.

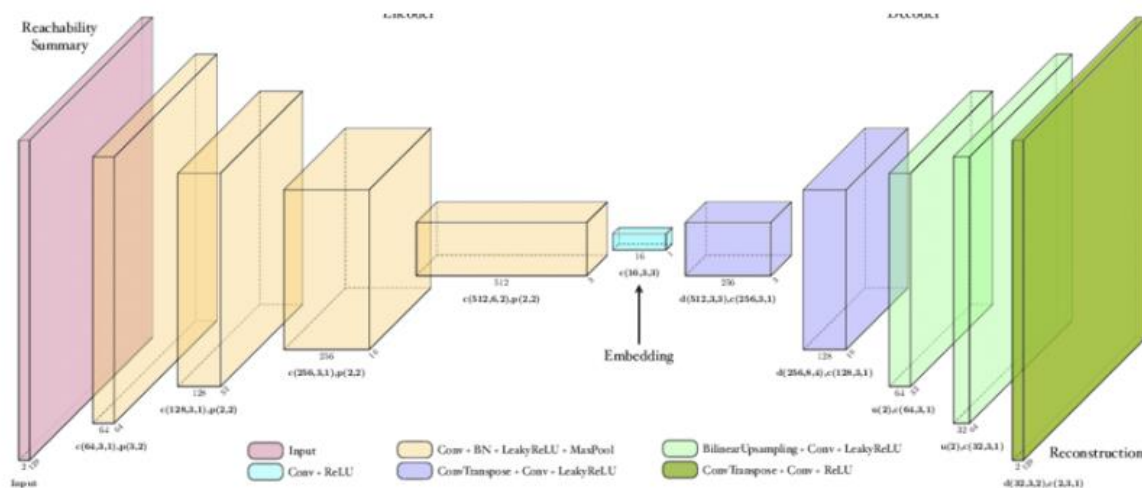
Contractive autoencoders (CAEs) are a type of neural network architecture used in unsupervised learning to learn a compressed representation of the input data.

Working of Contractive Autoencoders

A contractive autoencoder is less sensitive to slight variations in the training dataset. We can achieve this by adding a penalty term or regularizer to whatever cost or objective function the algorithm is trying to minimize. The result reduces the learned representation's sensitivity towards the training input. This regularizer needs to conform to the Frobenius norm of the Jacobian matrix for the encoder activation sequence concerning the input.

If this value is zero, we don't observe any change in the learned hidden representations as we change input values. But if the value is huge, then the learned model is unstable as the input values change.

We generally employ Contractive autoencoders as one of several other autoencoder nodes. It is in active mode only when other encoding schemes fail to label a data point.



Difference b/w Auto encoders and Contractive autoencoders:

The main difference between a CAE and a traditional autoencoder is that CAEs are trained with an additional regularization term that penalizes the model for small changes in the input data. This regularization term is added to the loss function during training and is designed to encourage the model to learn a more robust encoding of the input data.

The intuition behind this regularization term is that a good encoding should be invariant to small changes in the input data. For example, if the input data is an image of a face, a good encoding should not change significantly if the lighting conditions or facial expression change slightly. By penalizing the model for small changes in the input data, the CAE is encouraged to learn an encoding that is invariant to these kinds of variations.

The regularization term used in CAEs is called the contractive term and is defined as the Frobenius norm of the Jacobian matrix of the encoding with respect to the input. Intuitively, the Jacobian matrix measures the sensitivity of the encoding to changes in the input, so the Frobenius norm of this matrix gives a measure of how much the encoding changes in response to small changes in the input.

During training, the CAE is optimized to minimize a combination of the reconstruction error (i.e., the difference between the input and output of the network) and the contractive term. This encourages the model to learn a compressed representation of the input data that is both faithful to the original data and invariant to small changes in the input.

Advantages of CAEs:

1. Robust Feature Learning:

- CAEs are designed to learn features that are more robust to small variations in the input data. This makes the encoding more stable and reliable, which can be beneficial for downstream tasks like classification and regression.

2. Interpretability:

- The contractive penalty encourages the model to learn a smoother, more meaningful representation of the data, making it easier to interpret the learned features. This can lead to better generalization in machine learning tasks.

3. Dimensionality Reduction:

- CAEs are effective at reducing the dimensionality of input data while preserving important features. This is particularly useful in tasks where the original data is high-dimensional, such as image processing.

4. Improved Performance in Supervised Learning:

- The features learned by CAEs can be used as input to supervised learning models (e.g., classifiers or regressors), often leading to improved performance compared to using raw data.

Applications of CAEs:

1. Image Denoising:

- CAEs can effectively remove noise from images by learning a representation that captures the essential features of the image while ignoring noise.

2. Image Compression:

- By learning a lower-dimensional representation of images, CAEs can be used for image compression, reducing the amount of storage required for images without significant loss of quality.

3. Feature Extraction:

- CAEs can be used to extract meaningful features from data, which can then be used in other machine learning models for tasks such as classification or clustering.

4. Data Generation:

- CAEs can be used in generative tasks to convert data from one form to another, such as generating new images based on learned features.

5. Preprocessing for Supervised Learning:

- CAEs can serve as a preprocessing step to transform raw data into a more suitable form for supervised learning tasks, often leading to better model performance.

LANGUAGE MODELLING

Language modeling is a fundamental task in natural language processing (NLP) and deep learning, where the goal is to predict the next word (or sequence of words) in a sentence given the previous words. This task is crucial for many applications, including text generation, machine translation, speech recognition, and more.

Language models are used in a variety of NLP tasks, such as speech recognition, machine translation, and text summarization.

Content generation. One of the areas where language models shine the brightest is content generation. This includes generating complete texts or parts of them based on the data and terms provided by humans. Content can range from news articles, press releases, and blog posts to online store product descriptions, poems, and guitar tabs, to name a few.

Part-of-speech (POS) tagging. Language models have been widely used to achieve state-of-the-art results on POS tagging tasks. POS tagging is the process of marking each word in a text with its corresponding part of speech, such as noun, verb, adjective, etc. The models are trained on large amounts of labeled text data and can learn to predict the POS of a word based on its context and the surrounding words in a sentence.

Question answering. Language models can be trained to understand and answer questions with and without the context given. They can provide answers in multiple ways, such as by extracting specific phrases, paraphrasing the answer, or choosing from a list of options.

Text summarization. Language models can be used to automatically shorten documents, papers, podcasts, videos, and more into their most important bites. Models can work in two

ways: extract the most important information from the original text or provide summaries that don't repeat the original language.

Sentiment analysis. The language modeling approach is a good option for sentiment analysis tasks as it can capture the tone of voice and semantic orientation of texts.

Conversational AI. Language models are an inevitable part of speech-enabled applications that require converting speech to text and vice versa. As a part of conversational AI systems, language models can provide relevant text responses to inputs.

Machine translation. The ability of ML-powered language models to generalize effectively to long contexts has enabled them to enhance machine translation. Instead of translating text word by word, language models can learn the representations of input and output sequences and provide robust results.

TYPES OF LANGUAGE MODELS

Language models come in different types that can be put into two categories — statistical models and those based on deep neural networks.

Statistical language models

Statistical language models are a type of model that use statistical patterns in the data to make predictions about the likelihood of specific sequences of words. A basic approach to building a probabilistic language model is to calculate n-gram probabilities.

An *n-gram* is a sequence of words, where *n* is a number greater than zero. To make a simple probabilistic language model, you calculate the likelihood of different n-grams (word combinations) in a text. This is done by counting the number of times each word combination appears and dividing it by the number of times the previous word appears. This idea is based on a concept called the *Markov assumption*, which says that the probability of a word combination (the future) depends only on the previous word (the present) and not the words that came before it (the past).

There are different types of n-gram models such as

Traditional models that consider the probability of a word based on a fixed number of preceding words

- *unigrams* that evaluate each word independently;
- *bigrams* that consider the probability of a word given the previous word;
- *trigrams* that consider the probability of a word given the two previous words; and so on.

N-grams are relatively simple and efficient, but they do not consider the long-term context of the words in a sequence.

Neural Language Models (NLMs):

These models use deep learning techniques to capture more complex dependencies in the text. They can be categorized into:

- **Recurrent Neural Networks (RNNs):** RNNs, particularly Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs), are designed to handle sequential data and maintain a memory of previous words in the sequence.
- **Transformers:** The Transformer architecture, with its self-attention mechanism, has become the state-of-the-art for language modeling. Models like GPT (Generative Pretrained Transformer) and BERT (Bidirectional Encoder Representations from Transformers) are based on this architecture.

Contextualized Language Models:

- Modern language models, like BERT and GPT, generate context-aware word embeddings, meaning the representation of a word changes depending on its context within a sentence.

Transfer Learning in NLP:

- Pretrained language models, such as GPT, BERT, and T5, are first trained on large corpora in a self-supervised manner (e.g., predicting masked words or the next sentence) and then fine-tuned on specific tasks like sentiment analysis or machine translation.

SEQUENCE TO SEQUENCE LEARNING

Sequence-to-Sequence (Seq2Seq) learning is a powerful framework in deep learning used for tasks where input and output are sequences of different lengths, such as in machine translation, text summarization, and speech recognition. This architecture is essential for converting a sequence from one domain to another, like translating a sentence in English to French.

Key Components of Seq2Seq Learning

1. Encoder-Decoder Architecture:

- The Seq2Seq model typically consists of two main parts: an **Encoder** and a **Decoder**.
- **Encoder:** The encoder processes the input sequence and compresses it into a fixed-length context vector (also called a hidden state or thought vector), which captures the information from the entire input sequence.
- **Decoder:** The decoder takes the context vector as input and generates the output sequence, one element at a time, until the end of the sequence is reached.

2. Recurrent Neural Networks (RNNs):

- Initially, Seq2Seq models were built using RNNs, such as LSTMs (Long Short-Term Memory) or GRUs (Gated Recurrent Units). These are well-suited for handling sequential data, where the order of data points matters.

3. Attention Mechanism:

- A significant advancement in Seq2Seq learning is the **Attention Mechanism**. Attention allows the decoder to focus on different parts of the input sequence at each step of the output generation, rather than relying on a single context vector. This improves performance, especially for longer sequences, by allowing the model to use information from the entire input sequence more flexibly.

4. Transformer Architecture:

- The introduction of the Transformer architecture has revolutionized Seq2Seq learning. Unlike RNNs, Transformers do not require sequential processing, making them much more efficient for training. Transformers use self-attention mechanisms to weigh the importance of different words in the input sequence, making them powerful for tasks like translation, summarization, and more.

How Seq2Seq Works:

1. Encoding:

- The input sequence (e.g., a sentence in English) is passed through the encoder. Each word or token is processed sequentially, updating the hidden state. The final hidden state is often used as the context vector that summarizes the input sequence.

2. Decoding:

- The decoder starts with the context vector and generates the first word of the output sequence (e.g., the first word of the translated sentence in French). This generated word is then fed back into the decoder to predict the next word, and this process continues until the end-of-sequence token is produced.

3. Training:

- Seq2Seq models are trained using pairs of input and output sequences. The model learns to minimize the difference between the predicted sequence and the actual target sequence by using loss functions like cross-entropy.

Use Cases of the Sequence to Sequence Models

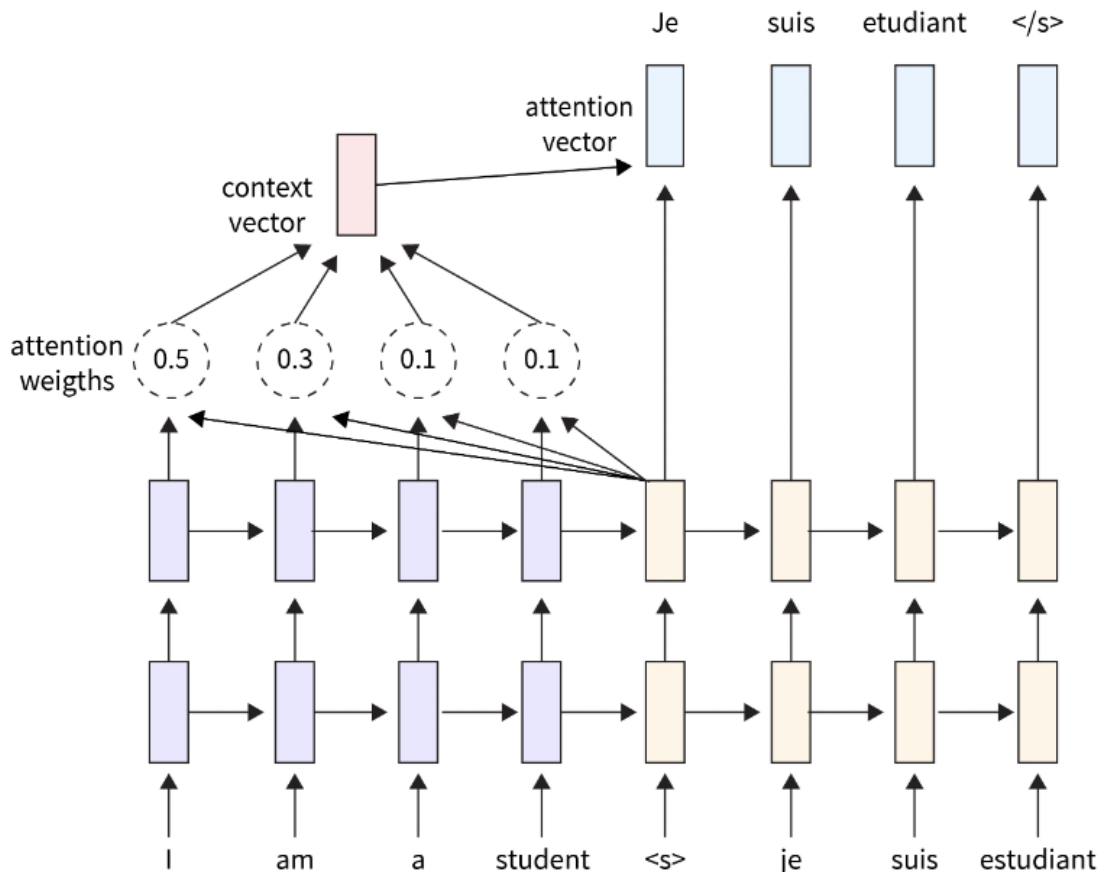
Many of the technologies you use every day are based on sequence-to-sequence models. For instance, voice-activated gadgets, online chatbots, and services like Google Translate are all powered by the sequence-to-sequence model architecture. Among the applications are the following:

- **Machine translation** - The seq2seq model predicts a word from the user's input, after which each subsequent word is predicted using the likelihood that the first word will appear.
- **Video captioning** - To understand the temporal structure of the sequence of frames and the sequence model of the generated sentences with RNNs, a sequence-to-sequence model for Video captioning was developed.

- **Text summarization** - Using neural sequence-to-sequence models, an effective novel method for abstractive text summarization has been made available (not restricted to selecting and rearranging passages from the original text)

ENCODER-DECODER ARCHITECTURE

Both have the Decoder on the right and the Encoder on the left. The Encoder and decoder output are combined to predict the following word at each time step.



One sequence is converted into another via sequence to sequence model (sequence transformation). It uses a recurrent neural network (RNN) or, more frequently, an LSTM or GRU to get over the vanishing gradient issue. The output from the preceding stage serves as the context for each item. One Encoder and one decoder network make up the main parts. First, the Encoder converts each item into a corresponding hidden vector that includes the item and its context. Then, with the previous output as the input context, the Decoder reverses the process and produces the vector as an output item.

- **Encoder-** Transforms the input words into matching hidden vectors using deep neural network layers. Each vector represents the current word and its context.
- **Decoder-** To create the following hidden vector and ultimately predict the following word, it uses the current word, its hidden states, and the hidden vector created by the Encoder as input. Apart from these two, numerous optimizations must result in the following additional sequence-to-sequence model components.

- **Attention-** The input to the Decoder is a single vector that must contain all of the context-related data. Large sequences cause difficulty with this. As a result, the attention mechanism lets the decoder focus on specific input sequences and generate an attention vector.
- **Beam Search-** The Decoder chooses the word with the highest probability as the output. But because greedy algorithms are the root of the issue, this does not necessarily produce the optimal outcomes. As a result, beam search is used, suggesting potential translations at each stage. A tree of the top k-results is created to do this.
- **Bucketing-** Because both the input and the output of a sequence-to-sequence model are padded with zeroes, variable-length sequences are conceivable. However, if we put the maximum length at 100 and the sentence is just three words long, there will be a significant loss of space. So, we apply the bucketing concept. First, we create buckets of varying sizes, such as (4, 8), (8, 15), and so forth, where 4 is the maximum input length and 8 is the maximum output length.

SPEECH RECOGNITION

Speech recognition, also known as automatic speech recognition (ASR), is the process of converting spoken language into text. Deep learning has significantly advanced the field, enabling highly accurate and efficient ASR systems that are widely used in virtual assistants, transcription services, and voice-controlled applications.

SPEECH RECOGNITION ALGORITHMS

Speech recognition technology relies on complex algorithms to translate spoken language into text or commands that computers can understand and act upon. Here are the algorithms and approaches used in speech recognition:

1. Hidden Markov Models (HMM)

Hidden Markov Models have been the backbone of speech recognition for many years. They model speech as a sequence of states, with each state representing a phoneme (basic unit of sound) or group of phonemes. HMMs are used to estimate the probability of a given sequence of sounds, making it possible to determine the most likely words spoken. Although newer methods have surpassed HMM in performance, it remains a fundamental concept in speech recognition, often used in combination with other techniques.

Speech as a Sequence of States

In the context of speech recognition, the primary task is to convert an acoustic signal (sound) into a sequence of words. Speech can be naturally represented as a sequence of phonemes, which are the smallest units of sound in a language. For example, the word "cat" is made up of three phonemes: /k/, /æ/, and /t/. An HMM models this sequence by assigning each phoneme or group of phonemes to a "state" in the model.

Structure of an HMM

An HMM is defined by the following components:

1. **States:** These represent the possible phonemes or groups of phonemes in the speech signal. Each state corresponds to a specific phoneme.
2. **Transition Probabilities:** These probabilities define the likelihood of transitioning from one state (phoneme) to another. For example, there might be a high probability of moving from the state representing /k/ to the state representing /æ/ in the word "cat."
3. **Emission Probabilities:** These describe the likelihood of observing a particular sound (acoustic feature) given a specific state (phoneme). The observed acoustic signal is modeled as being generated by these hidden states.
4. **Initial Probabilities:** These define the likelihood of starting in a particular state (phoneme) at the beginning of the sequence.

How HMM Works in Speech Recognition

The goal of speech recognition using HMMs is to determine the most likely sequence of words given the observed acoustic signal. Here's a step-by-step outline of how HMMs are used:

1. **Acoustic Feature Extraction:** The speech signal is first converted into a sequence of acoustic features. These features are typically short-term spectral representations of the sound, such as Mel-Frequency Cepstral Coefficients (MFCCs).
2. **Modeling the Speech Signal:** The sequence of acoustic features is then compared to the HMM, which models different phonemes. Each state in the HMM corresponds to a specific phoneme or sound.
3. **Viterbi Algorithm:** This algorithm is used to find the most likely sequence of hidden states (phonemes) that could have produced the observed sequence of acoustic features. This sequence of states is then mapped to the corresponding words.
4. **Word Recognition:** Once the sequence of phonemes has been identified, it is mapped to words using a pronunciation dictionary, and the most likely sequence of words is output as the recognized speech.

2. Natural language processing (NLP)

NLP is the area of artificial intelligence which focuses on the interaction between humans and machines through language through speech and text. Many mobile devices incorporate speech recognition into their systems to conduct voice search. Example such as: Siri or provide more accessibility around texting.

3. Deep Neural Networks (DNN)

DNNs have improved speech recognition's accuracy a lot. These networks can learn hierarchical representations of data, making them particularly effective at modeling complex patterns like those found in human speech. DNNs are used both for acoustic modeling, to better understand the sound of speech, and for language modeling, to predict the likelihood of certain word sequences.

One of the key strengths of DNNs is their ability to learn hierarchical representations. In the context of speech recognition, this means that a DNN can process raw acoustic signals (like waveforms or spectral features) and transform them into increasingly abstract representations that are easier to interpret. This hierarchical structure is crucial because speech is inherently complex, involving various layers of information such as phonemes, syllables, words, and sentences.

- **Lower Layers:** These layers capture basic features of the sound, such as frequency components, energy levels, and other acoustic features.
- **Middle Layers:** These layers begin to capture more complex patterns, such as phonemes or short sequences of sounds.
- **Higher Layers:** These layers represent even more abstract concepts, like entire words or phrases, and can capture contextual information that is critical for understanding speech in its entirety.

By learning these hierarchical representations, DNNs can effectively model the complexities of speech, which include variations in pitch, tone, speed, and accents.

DNNs for Acoustic Modeling

Acoustic modeling is a core component of speech recognition. It involves representing the relationship between the audio signal and the phonetic units (such as phonemes) that make up speech. Traditionally, Hidden Markov Models (HMMs) were used for this task, but DNNs have largely taken over due to their superior performance.

In acoustic modeling, the role of the DNN is to map the raw acoustic features (e.g., Mel-Frequency Cepstral Coefficients or MFCCs) to a probability distribution over phonemes. This mapping allows the system to predict which phoneme is most likely given the current segment of audio.

- **Learning from Data:** DNNs are trained on large datasets of labeled speech, where the input is the acoustic signal, and the output is the corresponding phoneme or sequence of phonemes. The network learns to identify patterns in the audio that correspond to specific phonemes, even in the presence of noise, different accents, or variations in speech speed.
- **Dealing with Variability:** One of the advantages of DNNs is their ability to generalize across different speakers and environments. They can learn to recognize the same phoneme even when it is pronounced differently by different people or in different contexts.

DNNs for Language Modeling

Language modeling involves predicting the likelihood of a sequence of words. In speech recognition, language models help in determining the most probable sequence of words given a particular acoustic input. This is especially important for handling homophones (words that sound the same but have different meanings) and for ensuring that the recognized words form coherent sentences.

- **Word Sequences:** DNNs can be trained to predict the likelihood of a word following a given sequence of words. For example, in the sentence "I want to eat an," the word "apple" is more likely to follow than "airplane." The DNN captures these probabilities based on patterns it has learned from large text corpora.
- **Contextual Understanding:** DNN-based language models can also take into account longer contexts, which improves their ability to predict the next word in a sequence. This is in contrast to traditional n-gram models that only consider a fixed number of previous words.
- **Handling Ambiguity:** By using DNNs for language modeling, speech recognition systems can better handle ambiguities in speech. For instance, in noisy environments or when dealing with unclear pronunciations, the language model can use context to choose the most likely word.

Combined Use in Speech Recognition Systems

In modern speech recognition systems, DNNs are used for both acoustic modeling and language modeling, and often these tasks are integrated. This combination allows for more accurate and robust speech recognition.

- **End-to-End Models:** Some advanced systems use end-to-end models where a single DNN is trained to map the acoustic input directly to the text output. These models, such as those based on sequence-to-sequence architectures with attention mechanisms, have further pushed the boundaries of what is possible in speech recognition.
- **Hybrid Models:** In hybrid models, a DNN might be used in conjunction with an HMM, where the DNN handles the acoustic modeling and the HMM deals with the temporal dynamics of speech. This approach leverages the strengths of both models.

4. End-to-End Deep Learning

Now, the trend has shifted towards end-to-end deep learning models, which can directly map speech inputs to text outputs without the need for intermediate phonetic representations. These models, often based on advanced RNNs, Transformers, or Attention Mechanisms, can learn more complex patterns and dependencies in the speech signal.